

AI Navigators

Authors : Linn Pyae
Assistant : Khaing Zin Oo

April 2026

Chapter 1

Python Basics

1.1 Introduction to Python (Python အကြောင်း မိတ်ဆက်)

1.1.1 What is Python? (Python ဆိုတာ ဘာလဲ?)

Python is a popular, high-level programming language widely used across various fields of software development. Although Python is primarily known as an interpreted language, modern Python implementations utilize a compilation step into bytecode to enhance performance and support its extensive libraries. As a high-level language, Python's syntax is designed to be highly readable and closely resembles the human language.

Python ဆိုသည်မှာ ဆော့ဖ်ဝဲ တည်ဆောက်မှု (software development) နယ်ပယ် အသီးသီးတွင် ကျယ်ပြန့်စွာ အသုံးပြုနေသော လူကြိုက်များသည့် high-level programming language တစ်ခု ဖြစ်သည်။ Python ကို အဓိကအားဖြင့် သရုပ်ပြဘာသာစကား (interpreted language) တစ်ခုအဖြစ် အသိများသော်လည်း ခေတ်မီ Python စနစ်များသည် လုပ်ဆောင်ချက် ပိုမို မြန်ဆန်စေရန်နှင့် ၎င်း၏ ကြီးမားလှသော libraries များကို ပံ့ပိုးပေးနိုင်ရန်အတွက် bytecode အဖြစ် ပြောင်းလဲပေးသော အဆင့်အဖြစ်လည်း အသုံးပြုကြသည်။ High-level language တစ်ခုဖြစ်သည့် နှင့်အညီ Python ၏ ရေးထုံး/ စကားလုံးဝါကျ အထားအသို (syntax) သည် လူသားတို့ ပြောဆိုသော ဘာသာစကားနှင့် အနီးစပ်ဆုံး ဆင်တူပြီး ဖတ်ရလွယ်ကူစေရန် ဒီဇိုင်းထုတ်ထားသည်။

- **Web Development:** Powering the backend of robust websites and scalable applications. (ကောင်းမွန်ခိုင်ခံ့သော ဝဘ်ဆိုဒ်များနှင့် Scalable Applications များ၏ နောက်ကွယ်မှ စနစ်များကို မောင်းနှင်ပေးခြင်း။)
- **Discord Chat Bots:** Utilizing specialized libraries for seamless communication and automation. (အလိုအလျောက် ဆက်သွယ်ဆောင်ရွက်ပေးနိုင်သော စနစ်များအတွက် အထူးပြုလုပ်ထားသည့် libraries များကို အသုံးပြုခြင်း။)
- **Mathematics:** Performing complex scientific calculations and data analysis. (ရှုပ်ထွေးသိပ္ပံဆိုင်ရာ တွက်ချက်မှုများ (scientific calculations) နှင့် အချက်အလက်များကို ခွဲခြမ်းစိတ်ဖြာခင်း (Data Analysis) ။)
- **Machine Learning & AI:** Serving as the industry standard for developing intelligent systems. (ဉာဏ်ရည်တူစနစ်များ တည်ဆောက်ရာတွင် စံ ဘာသာစကားအဖြစ် အသုံးပြုခြင်း။)

1.1.2 Difference between compiled and interpreted languages (Compiled နှင့် Interpreted Language များ၏ ကွာခြားချက်)

Feature	Compiled Languages	Interpreted Languages
Translation (ဘာသာပြန်ဆိုခြင်း)	The entire source code is translated at once before execution. (ကုဒ်တစ်ခုလုံးကို အသုံးမပြုမီ တစ်ဆင့်တည်း ဘာသာပြန်ဆိုသည်။)	Code is translated and executed line-by-line during runtime. (ကုဒ်ကို လုပ်ဆောင် နေ စဉ် အတွင်း တစ်ကြောင်းချင်းစီ ဘာသာပြန်ပြီး အလုပ်လုပ်စေသည်။)
Output (ရလဒ်)	Creates a standalone executable file (e.g., .exe). (သီးခြား အသုံးပြု နိုင်သော ဖိုင်တစ်ခု (ဥပမာ- .exe ဖိုင်) ကို ထုတ်ပေးသည်။)	No separate executable; requires an interpreter program to run. (သီးခြား ဖိုင် မထုတ်ပေးဘဲ ကုဒ်ကို အလုပ်လုပ်စေရန် အတွက် interpreter ပရိုဂရမ် တစ်ခု လိုအပ်သည်။)
Performance (စွမ်းဆောင်ရည်)	Generally faster; the CPU executes native machine code directly. (ယေဘုယျ အားဖြင့် ပိုမို မြန်ဆန် လေ့ရှိသည်။ CPU သည် machine code ကို တိုက်ရိုက် အလုပ်လုပ် (run) စေနိုင်သည်။)	Generally slower; the interpreter adds overhead during execution. (ယေဘုယျ အားဖြင့် ပို နှေး သည်။ Interpreter သည် ကုဒ် ကို အလုပ်လုပ်စေရန် အဆင့်ဆင့် လုပ်ဆောင်ရသောကြောင့် ဖြစ်သည်။)
Portability (သယ်ဆောင်ရလွယ်ကူမှု/ ပေါ့ပါးဝန်ကျဉ်းမှု)	Platform-specific; must be recompiled for different operating systems. (အသုံးပြု မည့် စနစ် ပေါ်တွင် မူတည် သည်။ မ တူညီ သော operating system များ အတွက် ထပ်မံ ဘာသာပြန် (re-compile) ရန် လိုအပ် သည်။)	Highly portable; the same code runs on any system with a compatible interpreter. (မည်သည့် စနစ် တွင် မဆို အသုံးပြု နိုင်သည်။ တူညီသော ကုဒ်ကို တွဲဖက်သုံးနိုင်သော (compatible) interpreter ရှိ သည့် မည်သည့် စနစ် တွင် မဆို အသုံးပြု နိုင်သည်။)
Debugging (ကွန်ပျူတာ ပ ရှိ ဂရမ် မှ အပြစ် ကို ရှာဖွေဖော်ပြခြင်း)	Errors are caught during the "build" phase before the program runs. (ပရိုဂရမ် မစ တင် မီ build phase (တည်ဆောက် သည့် အဆင့်) တွင် error (အမှား) များ ကို ရှာဖွေ တွေ့ရှိ နိုင်သည်။)	Errors are discovered at runtime when the specific line is executed. (ပရိုဂရမ် အလုပ်လုပ်နေစဉ်အတွင်း သက်ဆိုင်ရာကုဒ်လိုင်းသို့ ရောက်ရှိမှသာ အမှားများကို တွေ့ရှိရသည်။)

1.1.3 Python Syntax compared to other programming languages (Python ၏ ရေးထုံးနှင့် အခြားသော Programming language များအား နှိုင်းယှဉ်ချက်)

Python was designed for readability, and has some similarities to the English language with influence from mathematics. Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses. Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python ကို ဖတ်ရလွယ်ကူ စေရန် ရည်ရွယ်ပြီး ဒီဇိုင်းထုတ်ထားသည်။ ၎င်းတွင် အင်္ဂလိပ်ဘာသာစကားနှင့် ဆင်တူသော လက္ခဏာရှိပြီး သင်္ချာဘာသာရပ်၏ လွှမ်းမိုးမှုများ ပါဝင်နေသည်။ အမိန့်တစ်ခု (command) ပြီးဆုံးကြောင်း ပြသရန် စာလုံးများ (semicolons သို့မဟုတ် parentheses) ကို အသုံးပြုသော အခြား programming language များနှင့် မတူဘဲ Python ကမူ စာကြောင်းအသစ် (new lines) ကိုသာ အသုံးပြုသည်။ Python သည် အမိန့်တစ်ခုချင်း

စီ၏ နယ်ပယ်အတိုင်းအတာ (scope) ကို သတ်မှတ်ရန် indentation (စာကြောင်းအစတွင် ချန်ထားသော နေရာလွတ်) ကို အသုံးပြုသည်။ ၎င်းသည် loops၊ functions နှင့် classes များ၏ နယ်ပယ်ကို ခွဲခြားရန် အသုံးပြုခြင်း ဖြစ်သည်။ အခြားသော Programming language များတွင်မူ အဆိုပါ ရည်ရွယ်ချက်အတွက် တွန့်ကွင်း (curly-brackets) များကို အသုံးပြုလေ့ရှိကြသည်။

Quiz

- If you are a native speaker of a language and you are interacting with that language, are you an interpreter or a compiler? Why? (အကယ်၍ သင်သည် မိခင်ဘာသာစကားတစ်ခုကို ကျွမ်းကျင်စွာ ပြောဆိုနိုင်သူဖြစ်ပြီး ထိုဘာသာစကားဖြင့်ပင် တစ်ဦးနှင့်တစ်ဦး အပြန်အလှန် ပြောဆိုဆက်သွယ်နေသည်ဆိုပါက၊ သင့်ကိုယ်သင် interpreter တစ်ယောက်ဟု ယူဆမည်လား သို့မဟုတ် compiler တစ်ယောက်ဟု ယူဆမည်လား? ဘာကြောင့်လဲ?)

Homework

- If you listen to a math lecture in total silence and later return home to write notes based on your understanding, would you consider yourself an interpreter or a compiler? Why? (အကယ်၍ သင်သည် သင်္ချာဘာသာရပ် သင်ကြားမှုကို အသံတိတ် အာရုံစိုက်နားထောင်ပြီးနောက်၊ အိမ်ပြန်ရောက်မှ သင်နားလည်သလို မှတ်စုများ ပြန်လည်ရေးသားမည်ဆိုပါက၊ သင့်ကိုယ်သင် interpreter တစ်ယောက်ဟု သတ်မှတ်မည်လား သို့မဟုတ် compiler တစ်ယောက်ဟု ယူဆမည်လား? ဘာကြောင့်လဲ?)

1.2 Basic Syntax & Variables (အခြေခံ ရေးထုံးများနှင့် ကိန်းရှင်များ)

1.2.1 Executing Python Syntax (Python ရေးထုံးများကို execute လုပ်ခြင်း)

After installing the interpreter, Python syntax can be executed by writing directly in the Command Line as shown below:

Interpreter ကို ထည့်သွင်း(install)ပြီး နောက်၊ Python ရေးထုံးများ ကို အောက်တွင် ဖော်ပြထားသည့် အတိုင်း Command Line တွင် တိုက်ရိုက်ရေးသားပြီး စမ်းသပ်ကြည့်နိုင်ပါသည်။

Terminal

```
>>> print("Hello, World!")
Hello, World!
```

1.2.2 Statements

Since programming involves giving a computer program a list of **instructions** to **execute**, we refer to these instructions as **statements**. Normally, the interpreter or compiler runs the code line by line, as shown below:

Programming ရေးသားခြင်းဆိုသည်မှာ ကွန်ပျူတာ ပရိုဂရမ်တစ်ခုကို လုပ်ဆောင်ရမန် ညွှန်ကြားချက်များ (instructions) ပေးခြင်းဖြစ်ပြီး၊ ထိုညွှန်ကြားချက်များကို statements ဟု ခေါ်ဆိုသည်။ ပုံမှန်အားဖြင့် interpreter သို့မဟုတ် compiler သည် ကုဒ်များကို အောက်ပါအတိုင်း တစ်ကြောင်းချင်းစီ ဖတ်ပြီး လုပ်ဆောင်သည်။

... main.py

```
1 print("Hello, World!")
2 print("Have a good day.")
3 print("Learning Python is fun!")
```

Terminal

```
> Hello, World!  
> Have a good day.  
> Learning Python is fun!
```

1.2.3 Variables

Just like in mathematics, we can store data using a short name (such as x or y) or a more descriptive name (such as `age`, `car_name`, or `total_volume`). We can later use these variables in various places by referring to the variable name to access the data stored inside.

သင်္ချာဘာသာရပ်ကဲ့သို့ Python တွင်လည်း ကိန်းဂဏန်း အချက်အလက်များကို x , y အစရှိသည်ဖြင့် ကိုယ်စားပြုအမည်တို့များဖြင့် ဖြစ်စေ၊ အချက်အလက်များကို `age`, `car_name` သို့မဟုတ် `total_volume` စသည့် ပိုမိုရှင်းလင်းသော အမည်များဖြင့် သိမ်းဆည်းထားနိုင်ပါသည်။ နောင်တွင် ထိုသိမ်းဆည်းထားသော အချက်အလက်များကို ပြန်လည်အသုံးပြုလိုပါက ၎င်းတို့အား ပေးထားသော အမည် (Variable name) ကို ခေါ်ယူရုံဖြင့် အလွယ်တကူ အသုံးပြုနိုင်ပါသည်။

... main.py

```
1 myName = "Linn"  
2 myAge = 22  
3 print(myName)  
4 print(myAge)  
5 print(f"My name is {myName} and I am {myAge} years old.")
```

Terminal

```
> Linn  
> 22  
> My name is Linn and I am 22 years old.
```

1.2.4 Line Continuation

Python generally expects one statement per line. However, for better readability, you can use the **line continuation operator** (`\`) to split a single logical line into multiple physical lines.

Python သည် ပုံမှန်အားဖြင့် instruction တစ်ခုကို တစ်ကြောင်းတည်းတွင်သာ ရေးသားရန် မျှော်လင့်ပါသည်။ သို့သော်လည်း ကုဒ်များကို ပိုမိုဖတ်ရလွယ်ကူစေရန်အတွက် line continuation operator (`\`) ကို အသုံးပြု၍ logical line တစ်ကြောင်းတည်းကို physical lines အများအပြားအဖြစ် ခွဲခြားရေးသားနိုင်ပါသည်။

... main.py

```
1 myName = "Linn"  
2 myAge = 22  
3 print(f"My name is {myName} and " \\  
4       f"I am {myAge} years old.")
```

Terminal

```
> My name is Linn and I am 22 years old.
```

1.3 Data Types

In programming, the data type is a fundamental concept. Depending on the data type stored in a variable, it can perform different operations. Python includes the following built-in data types by default:

Programming တွင် အချက်အလက် အမျိုးအစား (data type) ဆိုသည်မှာ အခြေခံကျသော အယူအဆတစ်ခု ဖြစ်သည်။ ကိန်းရှင် (variable) တစ်ခုအတွင်း သိမ်းဆည်းထားသော အချက်အလက် အမျိုးအစားပေါ်မူတည်ပြီး မတူညီသော လုပ်ဆောင်ချက် (operations) များအား စွမ်းဆောင်နိုင်သည်။ Python တွင် အခြေခံအားဖြင့် အောက်ပါ အချက်အလက် အမျိုးအစားများ ပါဝင်သည်။

Built-in Data Types

- **Text Type:** str
- **Numeric Types:** int, float, complex
- **Sequence Types:** list, tuple, range
- **Set Types:** set, frozenset
- **Mapping Type:** dict
- **Boolean Type:** bool
- **Binary Types:** bytes, bytearray, memoryview
- **None Type:** NoneType

... main.py

```
1 myName = "Linn" # str
2 myAge = 22 # int
3 randomNumber = 67.67 # float
4 mylist = ["apple", "banana", "cherry"] # list
5 mySet = {"apple", "banana", "cherry"} # set
6 myExistence = True # bool
7
8 mydict = {
9     "apple": 2,
10    "banana": 10,
11    "cherry": 7
12 } # dict
13
14 myBinaryData = b"Hello" # bytes
15 myGirlfriend = None # NoneType
```

Quiz

- Imagine you are a lecturer and you want to memorize your students' names (including duplicates). What data type would you use, and why? (သင်သည် ကထိကတစ်ယောက်ဖြစ်ပြီး သင့်ကျောင်းသားများ၏ အမည်များကို (နာမည်တူများ အပါအဝင်) မှတ်မိစေရန် လုပ်ဆောင်မည်ဆိုလျှင် မည်သည့် data type ကို

အသုံးပြုမည်နည်း? ဘာကြောင့်လဲ?)

- Regarding the previous scenario, if you also wanted to memorize the students' ages alongside their names, which data type would you use, and why? (အထက်ပါ အခြေအနေ တွင် ပ အမည် များ နှင့် အတူ ကျောင်းသားများ၏ အသက်ကိုပါ တွဲဖက်မှတ်သားလိုပါက မည်သည့် data type ကို အသုံးပြုမည်နည်း? ဘာကြောင့်လဲ?)

Homework

- In what scenario would you use the None type, and why? မည်သည့်အခြေအနေမျိုးတွင် None type ကို အသုံးပြုမည်နည်း? ဘာကြောင့်လဲ?

1.4 Operators

1.4.1 Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations.

ကိန်းဂဏန်း တန်ဖိုး များ ဖြင့် အခြေခံ သင်္ချာ တွက်ချက် မှု များ ပြုလုပ် ရန် အတွက် Arithmetic operators များ ကို အသုံးပြုပါသည်။

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor Division	x // y

••• arithmetic.py

```
1 x = 10
2 y = 3
3
4 print(f"Addition: {x + y}")
5 print(f"Subtraction: {x - y}")
6 print(f"Multiplication: {x * y}")
7 print(f"Division: {(x / y):.2f}")
8 print(f"Modulus: {x % y}")
9 print(f"Exponentiation: {x ** y}")
10 print(f"Floor Division: {x // y}")
```

Terminal

```
> Addition: 13
> Subtraction: 7
> Multiplication: 30
> Division: 3.33
> Modulus: 1
> Exponentiation: 1000
> Floor Division: 3
```

1.4.2 Logical Operators

Logical operators are used to combine conditional statements. They result in either True or False.

Logical operators များကို အခြေအနေ တစ်ခု ထက် ပိုသော အဆိုပြုချက် (conditional statements) များကို ပေါင်းစပ်ရန် အသုံးပြုပါသည်။ ၎င်းတို့၏ ရလဒ်သည် True သို့မဟုတ် False ဟူ၍သာ ထွက်ပေါ်မည်ဖြစ်သည်။

- **and**: Returns True if both statements are true.
- **or**: Returns True if at least one statement is true.
- **not**: Reverse the result (Returns False if the result is true).

••• logic.py

```
1 a = 10
2 b = 5
3
4 # Using 'and'
5 print(a > 0 and b > 0)
6
7 # Using 'or'
8 print(a > 10 or b == 5)
9
10 # Using 'not'
11 print(not(a == 10))
```

Terminal

```
> True
> True
> False
```

Quiz

What will be the output of the following?

အောက်ပါကုဒ်၏ ရလဒ် (output) မှာ အဘယ်နည်း။

••• quiz.py

```
1 print(0.1 + 0.2 == 0.3)
```


1.5 If Statements and Conditional Logic

1.5.1 What is Conditional Logic?

Conditional Logic using Comparison Operators

In programming, we use comparison operators to evaluate the relationship between two values. The result of these operations is always a `bool` (`True` or `False`). In Python, we use the following comparison operators:

Programming တွင် တန်ဖိုးနှစ်ခုကြားရှိ ဆက်သွယ်မှုကို စစ်ဆေးရန်အတွက် နှိုင်းယှဉ်ခြင်းအော်ပရေတာများ (comparison operators) ကို အသုံးပြုပါသည်။ ထိုသို့စစ်ဆေးခြင်း၏ ရလဒ်သည် အမြဲတမ်း `bool` (`True` သို့မဟုတ် `False`) သာ ထွက်ပေါ်မည်ဖြစ်သည်။ Python တွင် အောက်ပါ နှိုင်းယှဉ်ခြင်းအော်ပရေတာများကို အသုံးပြုပါသည် -

- **Equals:** `a == b`
- **Not Equals:** `a != b`
- **Less than:** `a < b`
- **Less than or equal to:** `a <= b`
- **Greater than:** `a > b`
- **Greater than or equal to:** `a >= b`

Note: In most programming languages like C and C++, these conditions being satisfied mean 1 while 0 means not being satisfied, but in Python, True and False are formal objects of the bool type. Furthermore, Python uses the concept of "Truthiness," where empty containers and the number 0 evaluate to False.

မှတ်ချက်။ ။ C နှင့် C++ ကဲ့သို့သော ပရိုဂရမ်မင်းဘာသာစကားအများစုတွင် အခြေအနေတစ်ခုမှန်ကန်ပါက ၁ (1) ဟု သတ်မှတ်ပြီး မှားယွင်းပါက ၀ (0) ဟု သတ်မှတ်လေ့ရှိသည်။ သို့သော် Python တွင်မူ `True` နှင့် `False` တို့သည် `bool` အမျိုးအစားဝင် တရားဝင် Object များဖြစ်ကြသည်။ ထို့အပြင် Python တွင် "Truthiness" ဟူသော အယူအဆကို အသုံးပြုထားပြီး၊ အထဲတွင် ဘာမှမရှိသော ကွန်တိနာ (empty containers) များနှင့် ကိန်းဂဏန်း ၀ တို့ကို `False` အဖြစ် သတ်မှတ်တွက်ချက်ပါသည်။

1.5.2 If Statement

The `if` statement evaluates a condition (an expression that results in `True` or `False`). If the condition is true, the code block inside the `if` statement is executed. If the condition is false, the code block is skipped. The syntax for a conditional statement in Python uses a colon (`:`) and indentation.

Before we look at the Python syntax, let's look at the underlying logic. You can read an `if` statement like a sentence as below:

`if` statement သည် ပေးထားသော အခြေအနေ (condition) တစ်ခုမှန်၊ မမှန် (`True` သို့မဟုတ် `False`) ကို စစ်ဆေးတွက်ချက်ပါသည်။ အကယ်၍ အခြေအနေမှန်ကန်ပါက `if` statement အတွင်းရှိ ကုဒ်များကို လုပ်ဆောင်မည်ဖြစ်ပြီး၊ အခြေအနေမှားယွင်းနေပါက ထိုကုဒ်များကို လုပ်ဆောင်ခြင်းမရှိဘဲ ကျော်သွားမည်ဖြစ်သည်။ Python တွင် conditional statement များအတွက် ရေးသားပုံစနစ် (syntax) မှာ ကော်လံ (`:`) နှင့် အကွာအဝေး (indentation) တို့ကို အသုံးပြုရပါသည်။

Python syntax ကို မလေ့လာမီ၊ ၎င်း၏ အရင်းခံ လုပ်ဆောင်ပုံယူတ္တိ (logic) ကို အရင်ကြည့်ကြပါစို့။ `if` statement တစ်ခုကို အောက်ပါအတိုင်း ဝါကျတစ်ကြောင်းကဲ့သို့ ဖတ်ရှုနိုင်ပါသည် -

Logic Flow

IF (some condition is true) THEN

Execute this specific block of code.

ELSE

Execute this alternative block of code instead.

In Python, the **Condition** is usually a comparison (like $x > 5$), and the **Then** part is indicated by an indented block.

Python တွင် **Condition** (အခြေအနေ) ဆိုသည်မှာ များသောအားဖြင့် နှိုင်းယှဉ်ချက်တစ်ခု (ဥပမာ- $x > 5$) ဖြစ်ပြီး၊ **Then** (ထို့နောက်) အပိုင်းကိုမူ ရှေ့သို့အကွာအဝေး (indentation) ခြားထားသော code block တစ်ခုဖြင့် ဖော်ပြလေ့ရှိပါသည်။

Equal If Statement

••• equal.py

```
1 x = 5
2 y = 5
3
4 if x == y:
5     print("x equals y")
6 else:
7     print("x doesn't equal y")
```

Terminal

```
> x equals y
```

Not Equal If Statement

••• notequal.py

```
1 x = 10
2 y = 5
3
4 if x != y:
5     print("x doesn't equal y")
6 else:
7     print("x equals y")
```

Terminal

```
> x doesn't equal y
```

1.5.3 Elif Statement

The `elif` keyword is short for “else if.” In Python, it is a way of saying: “if the previous conditions were not true, then try this specific condition instead.”

Before we look at the Python syntax, let’s look at the underlying logic. You can read an `elif` statement like a flow of decisions:

`elif` ဟူသော စကားလုံးမှာ `“else if”` ကို အတိုချောက် ရေးထားခြင်း ဖြစ်ပါသည်။ Python တွင် ၎င်းသည် “အကယ်၍ အရင်အခြေအနေများမှာ မမှန်ကန်ခဲ့လျှင်၊ ယခုဖော်ပြပါ အခြေအနေကို ထပ်မံစမ်းသပ်ကြည့်ပါ” ဟု ဆိုလိုခြင်း ဖြစ်ပါသည်။

Python syntax ကို မလေ့လာမီ၊ ၎င်း၏ အရင်းခံ လုပ်ဆောင်ပုံယူတ္တိ (logic) ကို အရင်ကြည့်ကြပါစို့။ `elif` statement တစ်ခုကို ဆုံးဖြတ်ချက်များ ချမှတ်ရာတွင် အောက်ပါအတိုင်း စီးဆင်းမှု (flow) တစ်ခုကဲ့သို့ ဖတ်ရှုနိုင်ပါသည်။

Logic Flow

IF (Condition A is true) **THEN**
Execute Code Block A.
ELSE IF (Condition B is true) **THEN**
Execute Code Block B.
ELSE (None of the above are true) **THEN**
Execute the Final Fallback Code.

Example Usage of Elif Statement

••• elif.py

```
1 x = 10
2 y = 5
3
4 if x == y:
5     print("x equals y")
6 elif x < y:
7     print("x is less than y")
8 else:
9     print("x is greater than y")
```

Terminal

```
> x is greater than y
```

Quiz

Find the output results of the following problems.

Problem 1

••• problem01.py

```
1 x = 10
2 y = 5
3 z = 0
4
5 if y > z and x == y:
6     print("Statement 1 is true")
7 elif x > y and x > z:
8     print("Statement 2 is true")
9 else:
10    print("None of the statement was true")
```

Problem 2

```
••• problem02.py

1 x = 10
2 y = 5
3 z = 0
4
5 if y > z or x == y:
6     print("Statement 1 is true")
7 elif x > y or x > z:
8     print("Statement 2 is true")
9 else:
10    print("None of the statement was true")
```

Problem 3

```
••• problem03.py

1 x = 10
2 y = 5
3 z = 0
4
5 if y > z and x == y:
6     print("Statement 1 is true")
7 elif x < y or z > x:
8     print("Statement 2 is true")
9 else:
10    print("None of the statement was true")
```

1.6 Functions

1.6.1 What is a function?

A function is a block of code that **only** runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result and helps to reduce code repetition by making the logic reusable.

Function ဆိုသည်မှာ ၎င်းအား ခေါ်ယူအသုံးပြုသည့်အခါမှသာ အလုပ်လုပ်သော ကုဒ်အစုအဝေး (block of code) တစ်ခု ဖြစ်ပါသည်။ Function တစ်ခုအတွင်းသို့ parameters ဟု ခေါ်ဆိုသည့် အချက်အလက်များကို ပေးပို့နိုင်ပါသည်။ Function တစ်ခုသည် ရလဒ်တစ်ခုကို ပြန်လည်ပေးပို့နိုင်သလို (return)၊ လုပ်ဆောင်ချက်များကို တစ်ခါတည်း ရေးသားထားနိုင်ခြင်းကြောင့် ကုဒ်များကို ထပ်ခါတလဲလဲ ရေးသားရခြင်းမှ လျော့ချပေးပြီး ပြန်လည်အသုံးပြုနိုင်သော ယုတ္တိစနစ် (reusable logic) တစ်ခုဖြစ်အောင် ကူညီပေးပါသည်။

```
••• main.py

1 def my_function():
2     print("Hello from a function")
3
4 # Calling the function
5 my_function()
```

Terminal

```
> Hello from a function
```

1.6.2 Parameters vs. Arguments

Parameters

Definition of Parameter

The variable used as a **placeholder** within the function definition is called a parameter. These are defined during function creation, and you can include as many parameters as needed.

Function တစ်ခုကို စတင်တည်ဆောက်စဉ် (function definition) သတ်မှတ်လိုက်သော ကိန်းရှင် (variable) ကို parameter ဟု ခေါ်ဆိုပါသည်။ ၎င်းသည် တန်ဖိုးတစ်ခုအတွက် နေရာယူပေးထားသော placeholder တစ်ခုဖြစ်ပြီး၊ မိမိလိုအပ်သလောက် parameter အရေအတွက်ကို ထည့်သွင်းသတ်မှတ်နိုင်ပါသည်။

Arguments

Definition of Argument

The **actual value** passed to the function when it is invoked is called an argument. You must ensure the number of arguments matches the number of parameters defined in the function to avoid a `TypeError`.

Function တစ်ခုအား ခေါ်ယူအသုံးပြုသည့် အခါတွင် အမှန်တကယ် ထည့်သွင်းပေးလိုက်သော တန်ဖိုးကို argument ဟု ခေါ်ဆိုပါသည်။ `TypeError` မဖြစ်ပေါ်စေရန်အတွက် ထည့်သွင်းလိုက်သော argument အရေအတွက်သည် function တည်ဆောက်စဉ်က သတ်မှတ်ခဲ့သော parameter အရေအတွက်နှင့် ကိုက်ညီမှုရှိစေရန် သေချာစွာ ဂရုပြုရမည်ဖြစ်သည်။

... main.py

```
1 # x and y are parameters
2 def sum_values(x: int, y: int):
3     print(f"The sum of {x} and {y} is {x + y}")
4
5 # 6 and 7 are separate arguments
6 sum_values(6, 7)
```

Terminal

```
> The sum of 6 and 7 is 13
```

1.6.3 Return Statement

Definition of Return

The return statement is a vital programming concept that allows a function to communicate data back to the rest of the program. It exits the function and sends a specific value to the caller. Once a return statement is executed, the function immediately stops, and any subsequent code within that function is ignored.

return statement ဆိုသည်မှာ function တစ်ခုမှ အချက်အလက်များကို ပရိုဂရမ်၏ အခြားအစိတ်အပိုင်းများထံ ပြန်လည်ပေးပို့နိုင်ရန် လုပ်ဆောင်ပေးသော အရေးကြီးသည့် programming အယူအဆတစ်ခု ဖြစ်ပါသည်။ ၎င်းသည် function အတွင်းမှ ထွက်ခွာပြီး သတ်မှတ်ထားသော တန်ဖိုးတစ်ခုကို ခေါ်ယူသူ (caller) ထံသို့ ပေးပို့လိုက်ပါသည်။ return statement တစ်ခုကို လုပ်ဆောင်ပြီးသည်နှင့် ထို function သည် ချက်ချင်းရပ်တန့်သွားမည်ဖြစ်ပြီး၊ function အတွင်းရှိ နောက်ဆက်တွဲ ကုဒ်များကို လုပ်ဆောင်ခြင်းမရှိဘဲ ကျော်သွားမည်ဖြစ်သည်။

... main.py

```
1 # x and y are parameters
2 def sum_values(x: int, y: int):
3     return x + y # Sends the result back to the caller
4
5 first_val = 6
6 second_val = 7
7
8 # The function call is replaced by its returned value
9 total = sum_values(first_val, second_val)
10
11 print(f"The sum of {first_val} and {second_val} is {total}")
```

Terminal

```
> The sum of 6 and 7 is 13
```

Quiz

Find the output results of the following problem.

... quiz.py

```
1 def sum_values(x: int, y: int):
2     return x + y
3
4 def diff_values(x: int, y: int):
5     return x - y
6
7 def error():
8     print("The conditions were not satisfied.")
9
10 first_val = 10
11 second_val = 5
12 result = None
13
14 if first_val > second_val:
15     result = diff_values(first_val, second_val)
```

```

16     print(f"The difference is {result}.")
17 elif first_val < second_val:
18     result = sum_values(first_val, second_val)
19     print(f"The sum is {result}.")
20 else:
21     error()

```

Homework

Find the output results of the following problem.

```

... homework.py

1 first_val = 12
2 second_val = 5
3 func_return = None
4 result = None
5
6 def first_func(x :int, y: int):
7     func_return = modulus(x, y)
8     return 0 if func_return == 0 else second_func(x, y)
9
10 def second_func(x: int, y: int):
11     func_return = x / y
12     return func_return
13
14 def modulus(x: int, y: int):
15     return x % y
16
17 def error():
18     return "The conditions were not satisfied."
19
20 def main():
21     result = first_func(first_val, second_val) \
22         if first_val > second_val else error()
23     print(f"The output: {result}")
24
25 main()

```

References

[1] W3Schools. *Python Reference*. Retrieved from <https://www.w3schools.com/python>